Design: Student Management and Tracking Platform for Schools

Juan Pastrana
Facultad de Ingeniería
Universidad Nacional de Colombia
Bogotá DC, Colombia.
jpastranar@unal.edu.co

Nathalia Diaz
Facultad de Ingeniería
Universidad Nacional de Colombia
Bogotá DC, Colombia.
ndiazac@unal.edu.co

Sebastián Gordillo
Facultad de Ingeniería
Universidad Nacional de Colombia
Bogotá DC, Colombia.
jgordillom@unal.edu.co

October 23 2026

Contents

1 System Domain **5**
1.1 System Context . . . 5
1.1.1 Problem Statement . . . 5
1.1.2 System Purpose . . . 5
1.1.3 Scope . . . 5
1.1.4 System Boundaries and External Interfaces . . . 6
1.1.5 Technical and Design Constraints . . . 6
1.2 Actor Characterization . . . 6
1.2.1 System Actors (non-human) . . . 7
1.3 Business Rules . . . 8
1.3.1 Access and Security . . . 8
1.3.2 Academic Management and Grading . . . 8
1.3.3 Attendance . . . 9
1.3.4 Digital Observer . . . 9
1.3.5 Dropout Risk Classification . . . 10
1.3.6 Communication . . . 10

2 Business Model Processes **10**
2.1 Process Catalog . . . 11
2.2 BP-01: Attendance Registration and Dropout-Risk Detection . . . 11
2.3 BP-02: Grading Lifecycle . . . 13

3 Architecture Diagram **14**
3.1 Architectural Overview . . . 14
3.2 Technology Stack . . . 15
3.3 Layers . . . 17
3.4 Backend Components . . . 17
3.5 Communication Flows . . . 18
3.6 Quality Attributes Traceability . . . 19
3.7 Open Decisions . . . 20

4 CRC Cards **20**
4.1 Main Classes Identified . . . 20
4.2 CRC Cards . . . 21

5 Class Diagram **26**

6 Relational Database Model **27**
6.1 Table Definitions 28

7 Mockups **31**
7.1 Common Screens 31
7.1.1 Login 31
7.1.2 Role selection 31
7.2 Teacher Screens 32
7.2.1 Teacher Home 32
7.2.2 Daily Attendance 33
7.3 User inbox for messages, notifications, and received alerts 34
7.3.1 Inbox 34
7.4 Summary of the student's academic performance 35
7.4.1 Performance Summary 35

1 System Domain

1.1 System Context

1.1.1 Problem Statement

School dropout in primary and secondary education is driven by several factors, among which sustained low academic performance, irregular attendance, and early loss of motivation are the most decisive. Educational institutions struggle to centralize and analyze the data generated during the school day on time. The information systems currently available are complex, which pushes teachers back to manual, paper-based methods (attendance sheets, grade notebooks, paper observation records). As a result, warning signs are detected late, and guardians are rarely informed in time.

1.1.2 System Purpose

The system is a web platform that digitizes the daily academic workflow of a school (attendance, grading, student observations, and communication) and uses the accumulated data to **automatically classify each student's dropout risk** through a traffic-light scheme (Green, Yellow, Red). It notifies coordinators and guardians when the risk changes so that preventive action can be taken on time.

1.1.3 Scope

In scope:

- Authentication, password management, and role-based access control.
- Flexible evaluation schemes, grade entry (draft and published), and automatic average computation.
- Daily attendance registration, editing, and absence consolidation.
- Digital student observer (categorized annotations).
- Automated dropout-risk classification and special-condition tagging.
- Notifications, teacher–guardian messaging, and an institutional announcement board.
- Dashboards for coordinators, directors, guardians, and students.

Out of scope: tuition and financial management, official report-card (bulletin) generation for ministry reporting, timetable and classroom scheduling, and enrollment/admission processes.

1.1.4 System Boundaries and External Interfaces

- **Web client:** single-page application built with React and TypeScript, using `shadcn/ui` and Radix UI primitives; responsive for mobile, tablet, and desktop.
- **Backend:** Django REST API that holds the business logic; stateless authentication with JWT.
- **Persistence:** lightweight relational database accessed through the Django ORM.
- **Notification channel:** internal in-app notifications and, optionally, an external e-mail service for guardian alerts.

1.1.5 Technical and Design Constraints

- The architecture must keep presentation and business logic decoupled (frontend / backend).
- Common read and write operations must respond in less than 1.5 seconds on average.
- Student data must be protected: guardians and students can only access their own records.
- The interface must minimize steps for critical tasks (attendance, grading) so that teachers with low technical affinity can adopt it.
- Project documentation is maintained in Overleaf.

1.2 Actor Characterization

The system has seven roles within the educational community. The platform enforces what each role can see and do.

Actor	Description	Main interactions with the system
Teacher (subject teacher and course head teacher)	Faculty member responsible for one or more subjects and, optionally, the head teacher of a course. Often has limited technical experience.	Registers daily attendance and edits it within the allowed window; configures evaluation schemes; enters grades as draft and publishes them; records and reads observations; messages guardians and students; consults course statistics.

Actor	Description	Main interactions with the system
Coordinator	Staff member in charge of academic and disciplinary follow-up.	Monitors the risk dashboard; tags special conditions and repeating students; registers academic or disciplinary observations; generates alerts for at-risk students; sends notifications to guardians; approves or rejects grade modification requests.
Student	Enrolled learner. Consumes information; does not produce academic records.	Logs in; changes or resets password; consults grades, attendance history, observations, and announcements; accesses shared educational resources; sees own risk status.
Guardian	Legal tutor linked to one or more students.	Logs in; selects among linked students; views risk status and indicators (average, attendance, failed subjects, observations); reads grades and observations; receives notifications (risk change, published grades, absences, new observations); messages teachers.
Principal (*Rector*)	Highest institutional authority.	Publishes institutional announcements; consults institution-wide risk dashboards and indicators.
Director / Management staff (*Directivo*)	Senior academic or administrative staff.	Publishes announcements; consults the global risk dashboard with filters and aggregated indicators.
Administrator	Technical user in charge of platform configuration.	Manages users, roles, student–guardian links, courses, subjects, academic periods, and system parameters.

1.2.1 System Actors (non-human)

- **Risk classification engine:** internal process that re-evaluates a student's risk whenever attendance, published grades, or observations change.
- **Notification service:** internal (and optionally e-mail) service that delivers alerts to the corresponding users.

1.3 Business Rules

Rules marked with an asterisk (*) contain a default value or an assumption that should be confirmed with the institution and configured as a system parameter.

1.3.1 Access and Security

BR-01 Only registered and active users can log in, using their identifier/e-mail and password.

BR-02 Each session is managed with a JWT token; requests without a valid token must be rejected.

BR-03 Views, modules, and actions are restricted by role (Teacher, Coordinator, Student, Guardian, Principal, Director, Administrator).

BR-04 A guardian can only access records of the students formally linked to him or her; a student can only access his or her own records.

BR-05 A guardian must be linked to at least one student, and may be linked to several. A student may have more than one guardian.

BR-06 Password recovery must follow a guided verification flow, without administrator intervention.

1.3.2 Academic Management and Grading

BR-07 Each teacher defines the evaluation scheme (components and percentages) of each subject and period; the percentages of a scheme must add up to 100%.

BR-08 An academic activity may be split into several sub-evaluations, each with its own grade; the activity grade is derived from them according to the scheme.

BR-09 Grades must fall within the allowed numeric range (default 0.0–5.0*); out-of-range values are rejected.

BR-10 Grades are first saved as *draft* and are not visible to students or guardians.

BR-11 Grades become visible only after the teacher explicitly *publishes* the grades of a period.

BR-12 Averages and final grades per subject and student are calculated automatically using the weights and formulas of the evaluation scheme.

BR-13 A published grade can only be modified through a modification request that must be approved by a coordinator; if rejected, the grade remains unchanged.

BR-14 Every approved modification must keep an audit trail (previous value, new value, justification, requester, approver, date).

BR-15 A subject is considered failed when its final grade is lower than the passing threshold (default 3.0*).

1.3.3 Attendance

BR-16 There is one attendance record per student and per class/day, with a status (e.g., Present, Absent, Late, Excused).

BR-17 Attendance can be edited by the teacher only within 24 hours after it was registered, to correct errors or add justifications.

BR-18 When an absence is confirmed and is not justified, the system automatically notifies the student's guardians.

BR-19 The system consolidates absences and accumulated absence percentages by student, subject, and group.

1.3.4 Digital Observer

BR-20 Teachers and coordinators can register observations; each observation has an author, date, student, category (e.g., positive/negative, academic/behavioral/disciplinary), and description.

BR-21 Observations are shown in chronological order and are visible to authorized roles, including the student and his or her guardians.

BR-22 Registering a new observation notifies the student's guardians.

1.3.5 Dropout Risk Classification

BR-23 Risk is evaluated automatically and continuously from three inputs: accumulated attendance, academic performance, and frequency of observations.

BR-24 Each student always has exactly one current risk level: **Green** (no evident risk), **Yellow** (possible risk), or **Red** (high dropout risk).

BR-25 The thresholds and weights used by the algorithm are institutional parameters*, configurable by the administrator.

BR-26 Risk is recalculated every time attendance is registered or edited, grades are published or modified, or an observation is added.

BR-27 Whenever the risk level changes, the system generates an internal alert for coordinators and notifies the student's guardians; the change is stored in a risk history.

BR-28 Only coordinators can manually add context tags (special conditions, repeating year) to a student; these tags refine follow-up but do not replace the automatic classification.

1.3.6 Communication

BR-29 Messaging is bidirectional between a guardian and the teachers of his or her linked student; messages are stored and traceable.

BR-30 Teachers can also send official messages to their students.

BR-31 Only the Principal and Directors can publish announcements, which are visible to the whole educational community.

BR-32 Coordinators can send notifications to guardians about school events.

2 Business Model Processes

This section documents the core business processes of the platform as activity diagrams with swimlanes (one lane per actor or system component). Each process is traced to the functional requirements (RF), non-functional requirements (RNF) and business rules (BR). Notation: circle = start event; thick circle = end event;

rounded rectangle = activity; rectangle = sub-process; diamond = decision; diamond with "+" = parallel fork.

2.1 Process Catalog

ID	Process	Main actors	Business value
BP-01	Attendance registration and dropout-risk detection (**core**)	Teacher, risk engine, notification service, guardian, coordinator	Turns the daily school routine into an early-warning signal and informs guardians in time.
BP-02	Grading lifecycle (scheme, draft, publication, modification)	Teacher, coordinator, student, guardian	Guarantees consistent, auditable grades that feed the risk classification.

2.2 BP-01: Attendance Registration and Dropout-Risk Detection

Attribute	Description
Objective	Register the daily attendance of a course and, from it, re-evaluate the dropout risk of each student (Green, Yellow, Red), notifying guardians and coordinators when something relevant happens.
Trigger	The teacher opens the attendance screen of a course on a class day. The same recalculation flow is also triggered by attendance edits (within 24 h), grade publication or modification, and new observations (BR-26).
Actors	Teacher; Web client; Backend API; Risk classification engine; Notification service; Guardian; Coordinator.
Preconditions	The teacher is authenticated with a valid JWT and is assigned to the course; the academic period is active; students are enrolled in the course.
Postconditions	Attendance is stored (one record per student and day); the student has exactly one current risk level; any level change is stored in the risk history; guardians and coordinators have been notified when applicable.

Attribute	Description
Requirements	RF-01, RF-02, RF-12, RF-13, RF-14, RF-17 to RF-21, RF-24, RF-25; RNF-03, RNF-04, RNF-05.
Business rules	BR-02, BR-03, BR-16 to BR-19, BR-23 to BR-28.

Main flow.

1. The teacher selects the course and date; the web client loads the roster.
2. The teacher marks each student as Present, Absent, Late or Excused and submits (RF-12).
3. The web client sends the request to the backend with the JWT.
4. The backend validates token, role and payload (RF-01, RF-02, RNF-03). If invalid, the request is rejected and the client shows an error (alternate flow A1).
5. The backend persists the attendance record and one detail per student through the Django ORM (BR-16).
6. In parallel, (a) if an unjustified absence is confirmed, the notification service creates a notice for the guardians (RF-21); and (b) the backend triggers the risk recalculation (BR-26).
7. The risk engine gathers accumulated attendance, academic performance and observation frequency (RF-17), applies the institutional thresholds and weights (BR-25) and computes the traffic-light level (RF-18).
8. If the level changed, the engine stores the new level and a risk-history entry (BR-27); otherwise the current level is kept (alternate flow A2).
9. The notification service creates a visual internal alert for coordinators (RF-20) and notifies the guardians in-app (e-mail is an optional extension, see the technology section).
10. The coordinator reviews the dashboard and the alert (RF-24) and defines an intervention, optionally tagging a special condition or repeating year (RF-19). The guardian views the risk status and indicators (RF-25).

Alternate flows.

- **A1 – Invalid request:** token missing/expired, insufficient role or malformed payload; the backend answers with an error and nothing is stored.
- **A2 – No risk change:** the recalculation yields the same level; no history entry, alert or notification is produced.
- **A3 – Absence justified or excused:** no guardian notification is generated.

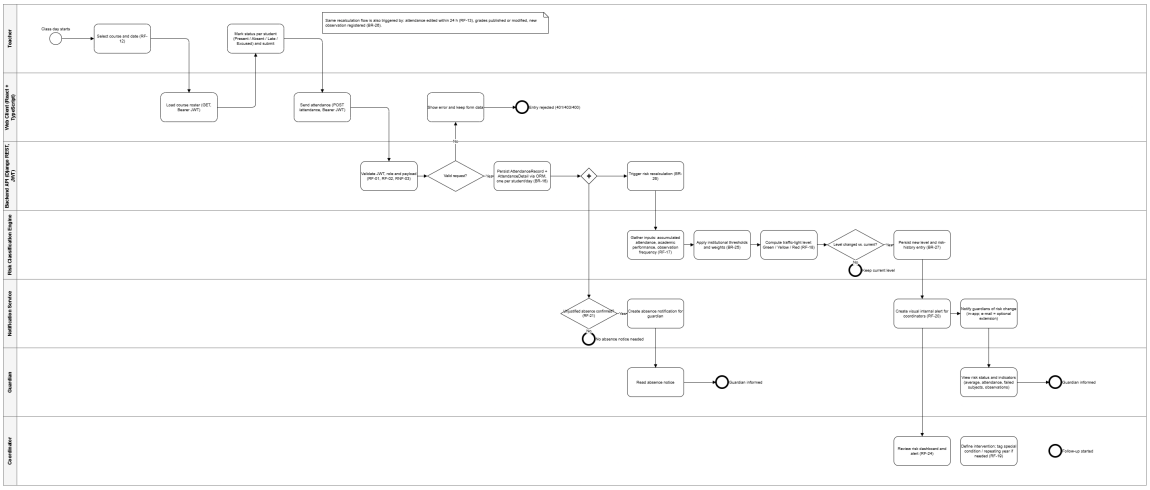


Figure 1: Activity diagram BP-01 – attendance registration and dropout-risk detection

2.3 BP-02: Grading Lifecycle

Attribute	Description
Objective	Define the evaluation scheme, enter grades as draft, publish them, and correct published grades through an approved and audited request.
Trigger	Start of an academic period (scheme definition); error detected in a published grade (modification).
Actors	Teacher, Backend API, Coordinator, Student, Guardian.
Preconditions	Authenticated teacher assigned to the subject; active academic period.
Postconditions	Published grades are visible to students and guardians; averages and final grades are computed; every approved modification has an audit trail.
Requirements	RF-05 to RF-11, RF-25; RNF-01, RNF-05.
Business rules	BR-07 to BR-15, BR-26, BR-27.

Main flow.

1. The teacher defines components, sub-evaluations and weights (RF-05, RF-06); the system checks that they add up to 100% (BR-07) and saves the scheme;

otherwise the teacher adjusts the weights.
2. The teacher enters grades individually or in bulk (RF-07); the system rejects values outside 0.0–5.0 (BR-09) and stores valid ones as *draft*, hidden from students and guardians (RF-08, BR-10).
3. The system computes provisional averages in real time for the teacher (RF-10); these remain invisible to students and guardians until publication.
4. The teacher publishes the grades of the period (RF-09, BR-11); grades and averages become visible to the linked community.
5. The risk recalculation sub-process (BP-01, steps 7–9) is executed and guardians and students are notified.
6. Students and guardians consult the published grades (RF-25).

Modification flow (RF-11). The teacher submits a modification request with justification (BR-13). The coordinator approves or rejects it. If approved, the system updates the grade, stores the audit trail (previous value, new value, justification, requester, approver, date; BR-14), recomputes the average and risk, and notifies the affected users. If rejected, the grade remains unchanged.

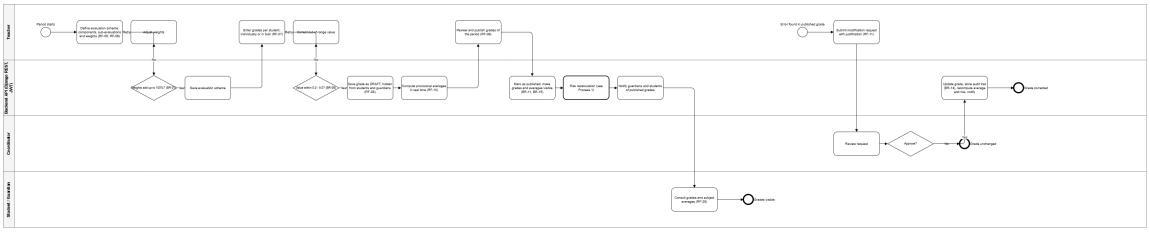


Figure 2: Activity diagram BP-02 – grading lifecycle

3 Architecture Diagram

3.1 Architectural Overview

The platform follows a **layered, client–server architecture** with the presentation and the business logic decoupled (RNF-07). A React + TypeScript single-page application consumes a stateless Django REST API authenticated with JWT (RNF-03); persistence goes through the Django ORM to a lightweight relational database (RNF-06). Shared educational resources are stored as files. An e-mail service is an optional extension for guardian alerts.

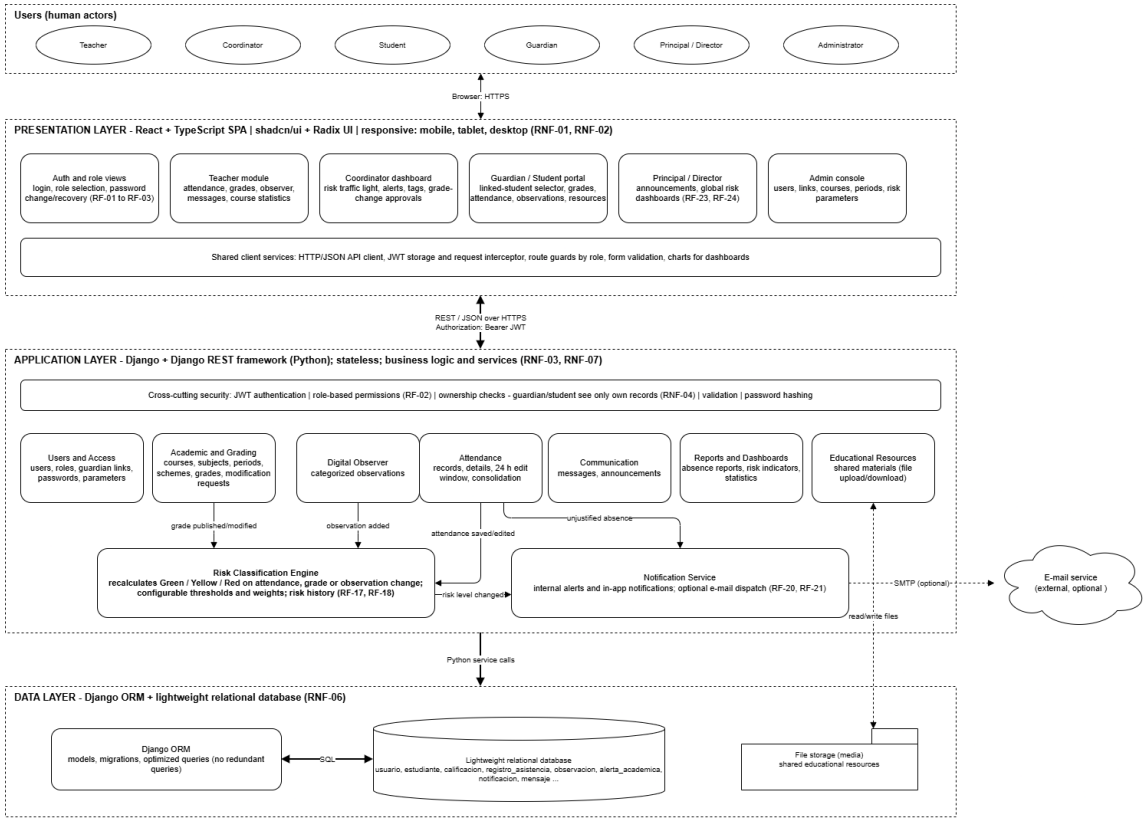


Figure 3: Software architecture diagram – layers, components, technologies and communication flows

3.2 Technology Stack

"Defined" means stated in the requirements; "Proposed" is a recommendation of this analysis; "To confirm" needs a team or institution decision.

Area	Technology	Source	Status	Rationale / note
Frontend framework	React with TypeScript (SPA)	RNF-02	Defined	Component-based, type-safe client.
UI components	`shadcn/ui` + Radix UI primitives	RNF-01	Defined	Accessible, low learning curve for teachers.
Responsiveness	Responsive layout (mobile, tablet, desktop)	RNF-02	Defined	Same SPA for all devices.

Area	Technology	Source	Status	Rationale / note
Build tool	Vite	–	Proposed	Standard toolchain for React + shadcn/ui.
Charts	shadcn chart components (Recharts)	RF-24, RF-25	Proposed	Dashboards and traffic-light indicators.
Backend	Django (Python)	RNF-03	Defined	Business logic and services.
API style	Django REST framework, REST/JSON	RNF-03, RNF-07	Proposed	Decoupled client–server contract.
Authentication	JWT tokens	RF-01, RNF-03	Defined	Stateless sessions. Library (e.g., `djangorestframework-simpl` and token lifetime: to confirm.
Authorization	Role-based permissions + ownership checks	RF-02, RNF-04	Defined	Enforced in the backend, never only in the client.
Persistence	Django ORM + lightweight relational database	RNF-06	Defined	Optimized models to avoid redundant queries. Engine: to confirm (see below).
File storage	Django media storage (local disk)	HU Student 9	Proposed	Needed for shared educational resources; no RF defined yet.
E-mail	SMTP via Django e-mail backend	–	To confirm	Not required by the requirements; in-app notifications are the baseline.
Documentation	Overleaf	RNF-08	Defined	Technical documentation only; not part of the runtime.
Deployment	Not defined	–	To confirm	Hosting, HTTPS certificate, backups and environments.

Database engine. The requirements only state "lightweight relational database". SQLite satisfies that description and is enough for a prototype or a single school, but it allows a single writer at a time, which may matter when many teachers submit attendance at the start of the day. Since the Django ORM abstracts the engine, the recommendation is to start with SQLite and keep PostgreSQL as the migration path; the decision should be confirmed with the institution.

Encryption. RNF-06 is titled "encrypted storage" but its text does not describe any encryption. This analysis assumes: HTTPS for data in transit, hashed passwords (Django default hashers), and encryption at rest delegated to the hosting/disk level, to be confirmed.

3.3 Layers

Layer	Technology	Responsibility
Presentation	React, TypeScript, `shadcn/ui`, Radix UI; responsive	Role-specific views (teacher, coordinator, guardian/student, principal/director, admin), form validation, route guards, API client with JWT handling, dashboard charts.
Application	Django, Django REST framework (Python); JWT	Business logic, authentication, role-based permissions, record-level ownership checks (RNF-04), risk classification, notification dispatch, reports.
Data	Django ORM; lightweight relational database; file storage	Persistence of the 17 entities of the relational model and of shared resource files; migrations and queries.
External	E-mail service (SMTP, optional)	Guardian alerts outside the platform.

3.4 Backend Components

Component	Requirements	Responsibility
Users and Access	RF-01 to RF-04	Users, roles, guardian–student links, password change and recovery, administration of courses, periods and system parameters.
Academic and Grading	RF-05 to RF-11	Courses, subjects, periods, evaluation schemes, draft/published grades, averages, modification requests with audit trail.
Attendance	RF-12 to RF-14	Attendance records and details, 24 h edit window, absence consolidation.
Digital Observer	RF-15, RF-16	Categorized observations, chronological listing.
Communication	RF-22, RF-23	Teacher–guardian messaging, institutional announcements.
Reports and Dashboards	RF-14, RF-24, RF-25	Aggregated absence reports, risk indicators, course statistics for each role's dashboard.
Educational Resources	HU Student 9	Upload and consultation of materials shared by teachers (requirement to be formalized).
Risk Classification Engine	RF-17 to RF-19	Recalculates Green/Yellow/Red when attendance, grades or observations change; configurable thresholds and weights; risk history.
Notification Service	RF-20, RF-21	Visual internal alerts, in-app notifications; optional e-mail dispatch.

3.5 Communication Flows

Flow	From → To	Protocol / format	Notes
F1	Browser ↔ Web client	HTTPS	Responsive access from desktop, tablet and mobile.
F2	Web client ↔ REST API	REST over HTTPS, JSON, `Authorization: Bearer <JWT>`	Stateless; requests without a valid token are rejected (RNF-03).

Flow	From → To	Protocol / format	Notes
F3	API modules → Risk engine	In-process calls	Triggered on attendance, grade or observation changes (BR-26).
F4	Risk engine / modules → Notification service	In-process calls	Risk change, unjustified absence, new observation.
F5	API ↔ Database	Django ORM / SQL	Common operations under 1.5 s on average (RNF-05).
F6	Resources module ↔ File storage	File I/O	Shared educational materials.
F7	Notification service → E-mail service	SMTP (optional)	Guardian alerts.

3.6 Quality Attributes Traceability

RNF	Requirement	Architectural response
RNF-01	Simplicity, low learning curve	`shadcn/ui` + Radix components; few steps for attendance and grading screens.
RNF-02	Responsive design	Single responsive React SPA.
RNF-03	Authentication and authorization	JWT on every API request; backend rejects invalid tokens.
RNF-04	Data protection and privacy	Role permissions plus ownership checks for guardians and students.
RNF-05	Response time < 1.5 s	Optimized ORM queries; synchronous risk recalculation limited to the affected student.
RNF-06	Persistence, optimized ORM	Lightweight relational DB via Django ORM; encryption approach to confirm.
RNF-07	Decoupled architecture	Separate React client and Django API communicating only through REST/JSON.
RNF-08	Documentation in Overleaf	Documentation outside the runtime architecture.

3.7 Open Decisions

- Database engine (SQLite vs. PostgreSQL) and encryption at rest.
- JWT library, access/refresh token lifetime and where the client stores the token.
- Whether the e-mail channel is in scope; if so, provider and templates.
- Formal requirement and storage limits for the educational resources module.
- Deployment topology: hosting, HTTPS, backups and environments.
- Risk recalculation is assumed synchronous inside the API process (service layer or Django signals) given school-scale load; revisit if response times degrade.

4 CRC Cards

This section identifies the main classes of the system For each class, a CRC card lists its responsibilities and the classes it collaborates with. The attributes and methods of each class are detailed in the class diagram Section.

4.1 Main Classes Identified

Class	ER table	Description
`User`	`usuario`	Registered person with credentials and a role; common ancestor of all human actors.
`Guardian`	`acudiente`	Guardian linked to one or more students; follows their academic progress.
`Student`	`estudiante`	Enrolled learner; central entity of the follow-up and risk detection.
`Teacher`	`docente`	Teaching staff member; may be subject teacher and/or homeroom teacher.
`Coordinator`	`coordinador`	Institutional staff member in charge of academic and disciplinary follow-up.
`Course`	`curso`	Group of students led by a homeroom teacher.
`Subject`	`asignatura`	Area of knowledge taught by one teacher in one course.
`AcademicPeriod`	`periodo_academico`	Time frame in which activities are evaluated.

Class	ER table	Description
`EvaluationActivity`	`actividad_evaluativa`	Weighted evaluative activity of a subject in a period.
`Grade`	`calificacion`	Score of a student in an evaluation activity.
`AttendanceRecord`	`registro_asistencia`	Attendance session of a course on a given date.
`AttendanceDetail`	`detalle_asistencia`	Attendance status of one student in one session.
`Observation`	`observacion`	Annotation in the student's digital observer.
`AcademicAlert`	`alerta_academica`	Alert raised for a student at academic or dropout risk.
`Notification`	`notificacion`	Alert or notice delivered to a user.
`Message`	`mensaje`	Official message exchanged through the system.
`MessageRecipient`	`destinatario_mensaje`	Delivery of a message to one recipient.

4.2 CRC Cards

Each card shows the class name (and its superclass, if any), its responsibilities on the left, and its collaborators on the right.

User	
Responsibilities	**Collaborators**
• Store identity, credentials, and role • Authenticate the user and manage the session (JWT) • Change and recover passwords • Enforce role-based access to views and actions	• Guardian • Student • Teacher • Coordinator • Message • MessageRecipient • Notification

Guardian *(extends User)*	
Responsibilities	**Collaborators**
• Know the students linked to the guardian • Let the guardian switch between linked students • Give access only to the linked students' records • Receive notifications about linked students	• User • Student • Notification • Message

Student *(extends User)*	
Responsibilities	**Collaborators**
• Hold course, guardian link, repeating flag, and special condition • Provide grades, attendance history, and observations • Supply the data used to detect academic risk	• User • Guardian • Course • Grade • AttendanceDetail • Observation • AcademicAlert

Teacher *(extends User)*	
Responsibilities	**Collaborators**
• Register and publish grades for assigned subjects • Take and correct attendance (homeroom teacher) • Register observations about students • Message guardians and students • Consult course statistics	• User • Course • Subject • Grade • AttendanceRecord • Observation • Message

Coordinator *(extends User)*	
Responsibilities	**Collaborators**
• Mark special conditions and repeating students • Register academic or disciplinary observations • Generate alerts for students at risk • Send notifications to the community	• User • Student • Observation • AcademicAlert • Notification

Course	
Responsibilities	**Collaborators**
• Group the students of a course • Know its homeroom teacher • List the subjects offered • Provide attendance sessions and statistics	• Student • Teacher • Subject • AttendanceRecord

Subject	
Responsibilities	**Collaborators**
• Link a course with its teacher • Know its evaluation activities • Compute the weighted average per student and period	• Course • Teacher • EvaluationActivity • Grade

AcademicPeriod	
Responsibilities	**Collaborators**
• Define start and end dates • Tell whether it is open or active • Group evaluation activities by period	• EvaluationActivity

EvaluationActivity	
Responsibilities	**Collaborators**
• Define the name and weight of the activity • Ensure the weights of a subject and period add up to 100% • Hold the grades given for the activity	• Subject • AcademicPeriod • Grade

Grade	
Responsibilities	**Collaborators**
• Store the score and its status • Validate that the value is in the allowed range • Stay hidden while in draft and become visible when published • Contribute to the subject average	• EvaluationActivity • Student • Teacher

AttendanceRecord	
Responsibilities	**Collaborators**
• Represent one attendance session of a course • Contain one detail per student • Provide absence totals	• Course • AttendanceDetail • Teacher

AttendanceDetail	
Responsibilities	**Collaborators**
• Store the status of a student in a session • Store the justification • Identify unjustified absences to notify the guardian • Feed the accumulated absence percentage	• AttendanceRecord • Student • Notification

Observation	
Responsibilities	**Collaborators**
• Store category, description, polarity, author, and date • Be listed chronologically per student • Trigger a notification to the guardian when registered	• Student • Teacher • Coordinator • Notification

AcademicAlert	
Responsibilities	**Collaborators**
• Record the reason and date of the alert • Track whether it is active • Support intervention and follow-up	• Student • Coordinator • Notification

Notification	
Responsibilities	**Collaborators**
• Store subject and content • Track delivery and read status • Notify users of risk changes, absences, grades, and observations	• User • Coordinator • Guardian

Message	
Responsibilities	**Collaborators**
• Store the sender, subject, content, and date • Manage its recipients • Keep communication formal and traceable	• User • MessageRecipient

MessageRecipient	
Responsibilities	**Collaborators**
• Track read status and confirmation per recipient • Link a message with each of its recipients	• Message • User

5 Class Diagram

This section presents the UML class diagram of the system, including attributes, methods, and relationships (generalization, association, composition, and multiplicity).

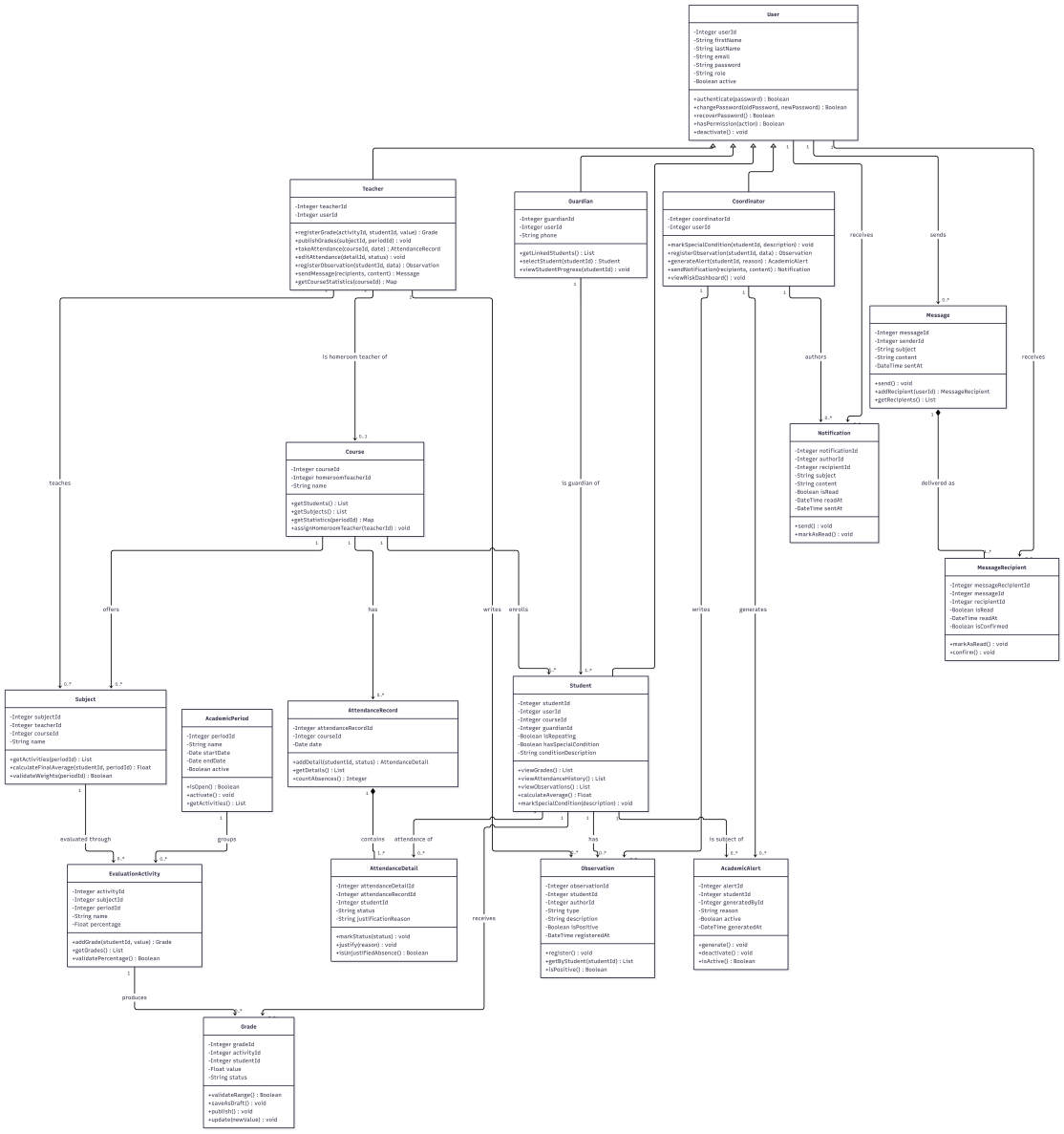


Figure 4: UML class diagram – general overview

6 Relational Database Model

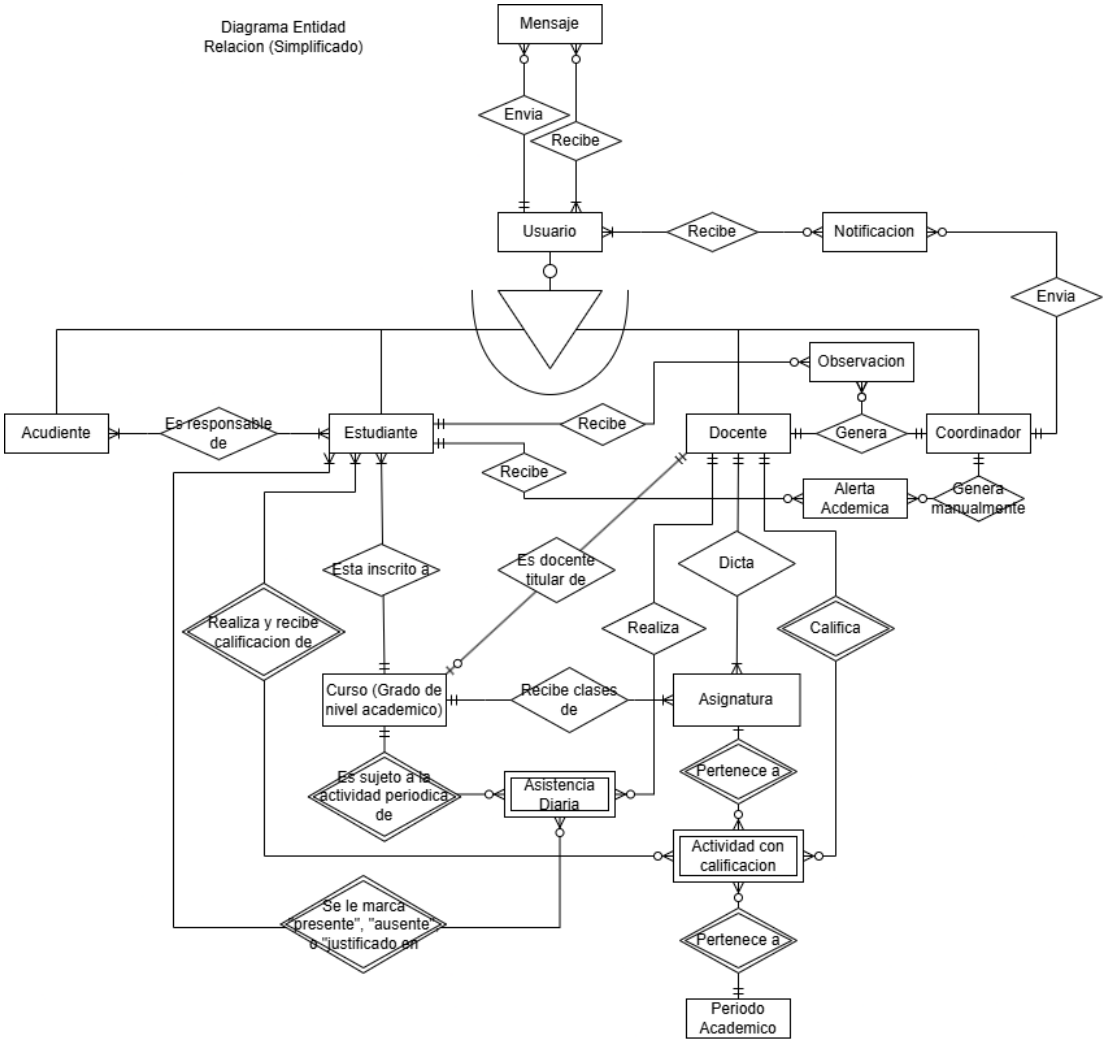


Figure 5: Entity–relationship diagram – simplified model

6.1 Table Definitions

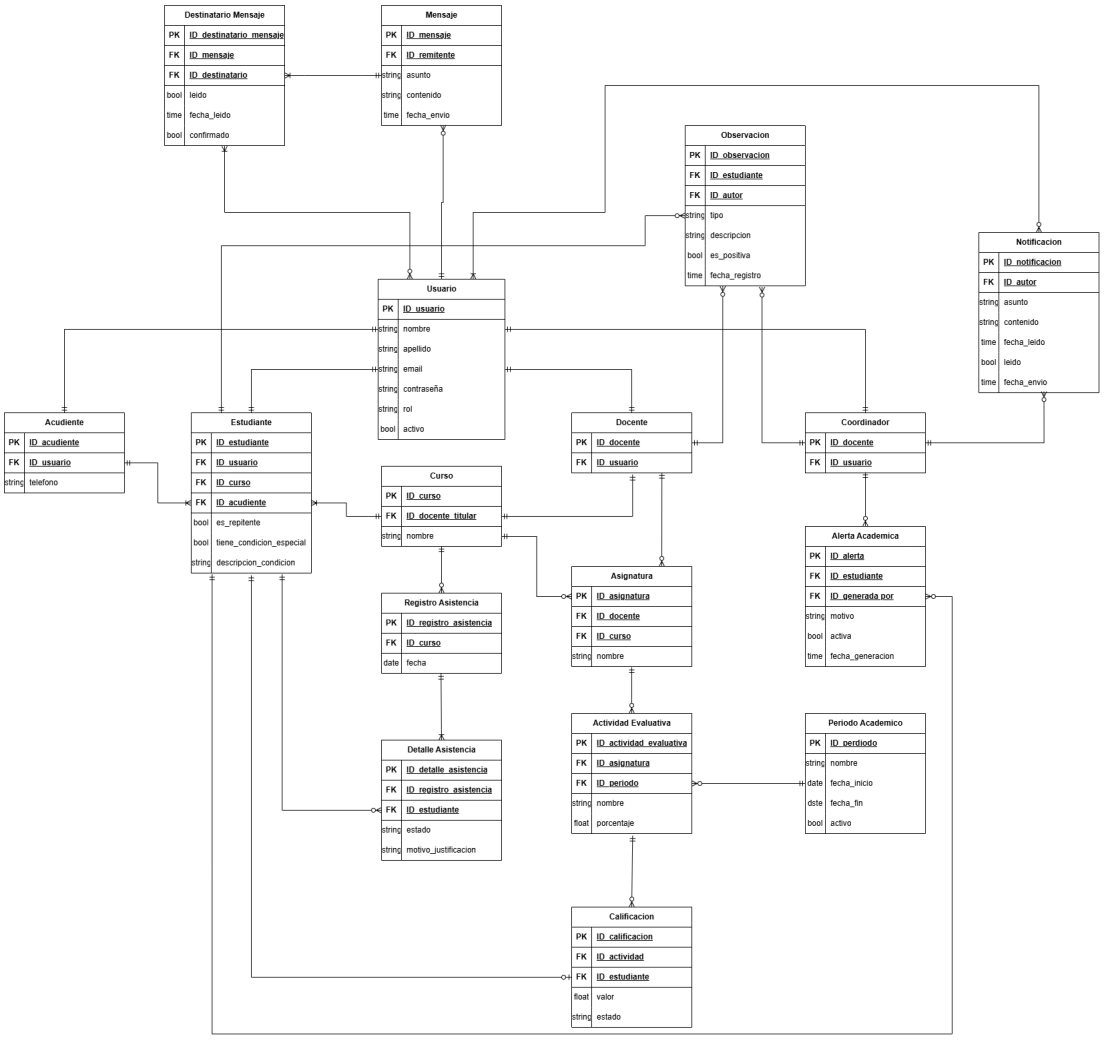


Figure 6: Entity–relationship diagram – full model

Table	Column	Type	Key / reference
usuario	`ID_usuario`	integer	PK
	`nombre`	string	
	`apellido`	string	
	`email`	string	
	`contraseña`	string	

Table	Column	Type	Key / reference
	rol	string	
	activo	bool	
acudiente	ID_acudiente	integer	PK
	ID_usuario	integer	FK → usuario
	telefono	string	
estudiante	ID_estudiante	integer	PK
	ID_usuario	integer	FK → usuario
	ID_curso	integer	FK → curso
	ID_acudiente	integer	FK → acudiente
	es_repitente	bool	
	tiene_condicion_especial	bool	
	descripcion_condicion	string	
docente	ID_docente	integer	PK
	ID_usuario	integer	FK → usuario
coordinador	ID_docente (sic)	integer	PK
	ID_usuario	integer	FK → usuario
curso	ID_curso	integer	PK
	ID_docente_titular	integer	FK → docente
	nombre	string	
asignatura	ID_asignatura	integer	PK
	ID_docente	integer	FK → docente
	ID_curso	integer	FK → curso
	nombre	string	
periodo_academico	ID_perdiodo	integer	PK
	nombre	string	
	fecha_inicio	date	
	fecha_fin	date	
	activo	bool	
actividad_evaluativa	ID_actividad_evaluativa	integer	PK
	ID_asignatura	integer	FK → asignatura
	ID_periodo	integer	FK → periodo_academico
	nombre	string	
	porcentaje	float	
calificacion	ID_calificacion	integer	PK
	ID_actividad	integer	FK → actividad_evaluativa
	ID_estudiante	integer	FK → estudiante
	valor	float	
	estado	string	

Table	Column	Type	Key / reference
registro_asistencia	ID_registro_asistencia	integer	PK
	ID_curso	integer	FK → curso
	fecha	date	
detalle_asistencia	ID_detalle_asistencia	integer	PK
	ID_registro_asistencia	integer	FK → registro_asistencia
	ID_estudiante	integer	FK → estudiante
	estado	string	
	motivo_justificacion	string	
observacion	ID_observacion	integer	PK
	ID_estudiante	integer	FK → estudiante
	ID_autor	integer	FK → docente / coordinador
	tipo	string	
	descripcion	string	
	es_positiva	bool	
	fecha_registro	time	
alerta_academica	ID_alerta	integer	PK
	ID_estudiante	integer	FK → estudiante
	ID_generada_por	integer	FK → coordinador
	motivo	string	
	activa	bool	
	fecha_generacion	time	
notificacion	ID_notificacion	integer	PK
	ID_autor	integer	FK → docente / coordinador
	ID_destinatario (proposed)	integer	FK → usuario
	asunto	string	
	contenido	string	
	leido	bool	
	fecha_leido	time	
	fecha_envio	time	
mensaje	ID_mensaje	integer	PK
	ID_remitente	integer	FK → usuario
	asunto	string	
	contenido	string	
	fecha_envio	time	
destinatario_mensaje	ID_destinatario_mensaje	integer	PK
	ID_mensaje	integer	FK → mensaje
	ID_destinatario	integer	FK → usuario
	leido	bool	

Table	Column	Type	Key / reference
	`fecha_leido`	time	
	`confirmado`	bool	

7 Mockups

7.1 Common Screens

7.1.1 Login

Purpose: Allows any registered user to authenticate with identifier/e-mail and password.

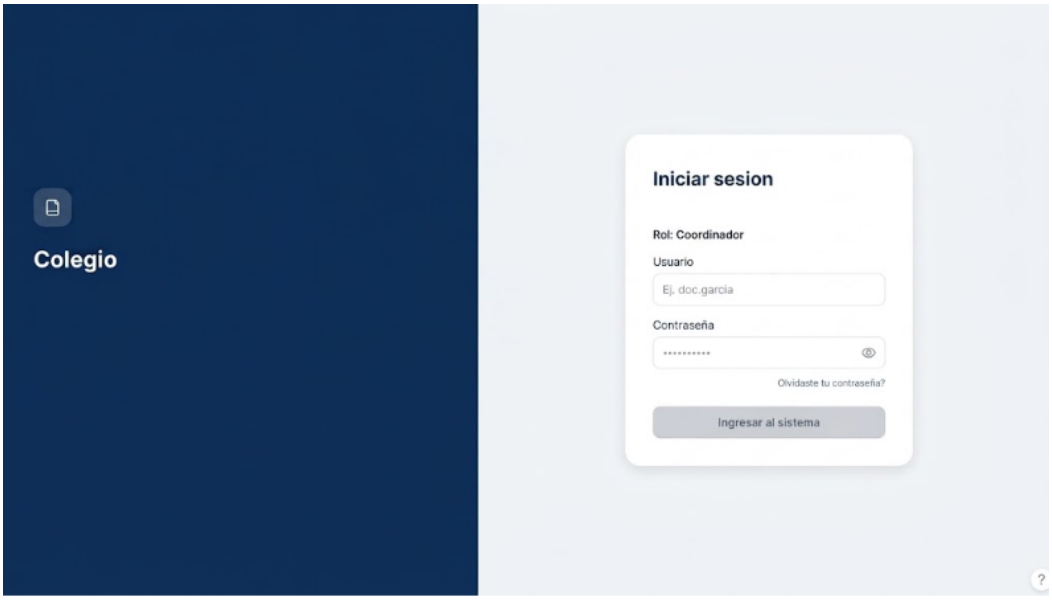


Figure 7: Mockup: Login

7.1.2 Role selection

Purpose: Lets users choose their role before starting.

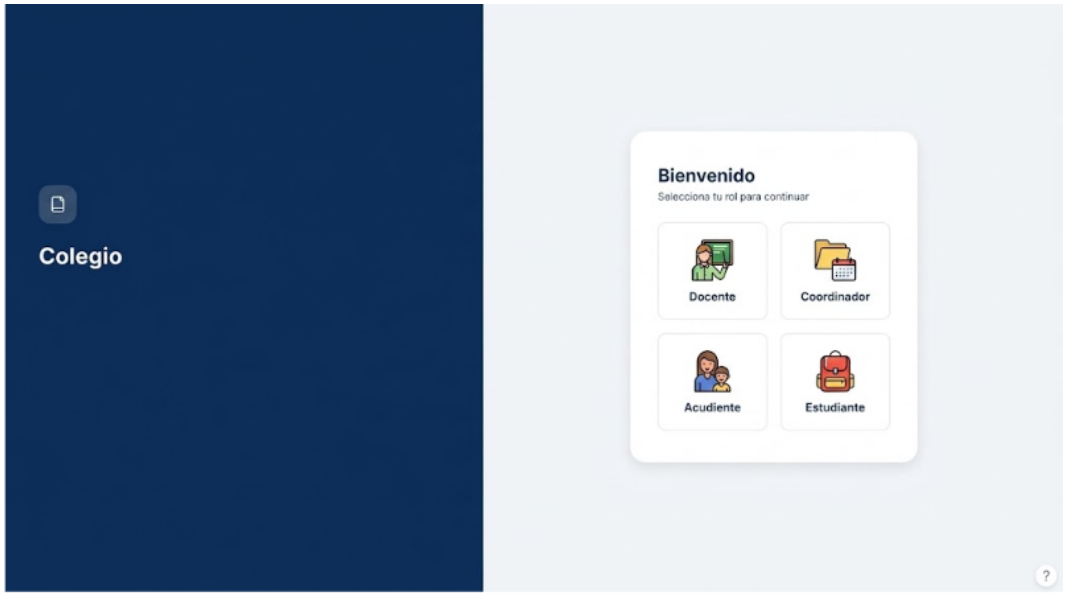


Figure 8: Mockup: Role selection

7.2 Teacher Screens

7.2.1 Teacher Home

Purpose: Entry point for the teacher: assigned courses and subjects, pending tasks, and quick access to attendance and grades.

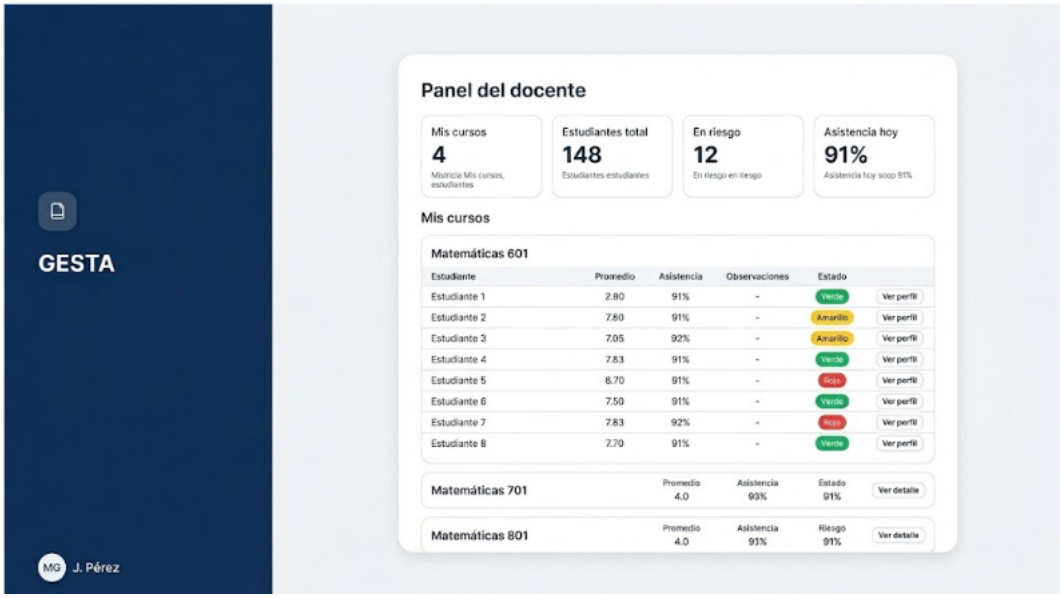


Figure 9: Mockup: Teacher Home

7.2.2 Daily Attendance

Purpose: Fast marking of each student's attendance status for a class, designed to take a few seconds.

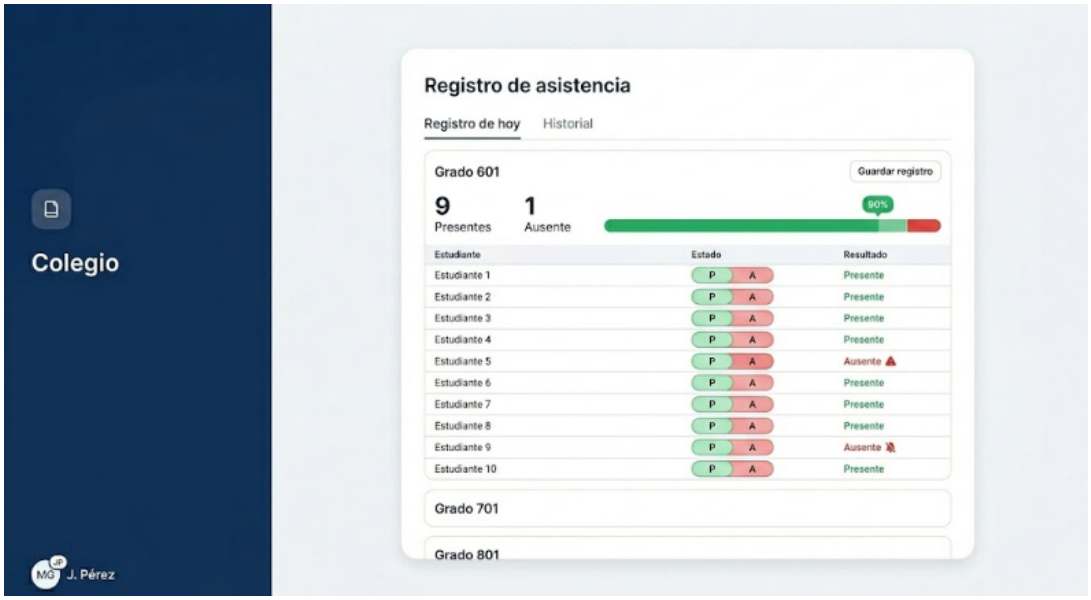


Figure 10: Mockup: Daily Attendance

7.3 User inbox for messages, notifications, and received alerts

7.3.1 Inbox

Purpose: Message page of users where nottifications and chats are displayed, they have their filters and features which can help to communicate with risky students more propperly.



Figure 11: Mockup: Inbox

7.4 Summary of the student's academic performance

7.4.1 Performance Summary

Purpose: A dashboard showing a summary of student performance, grades, and teacher comments—critical data to support mentorship and reduce dropout risks.

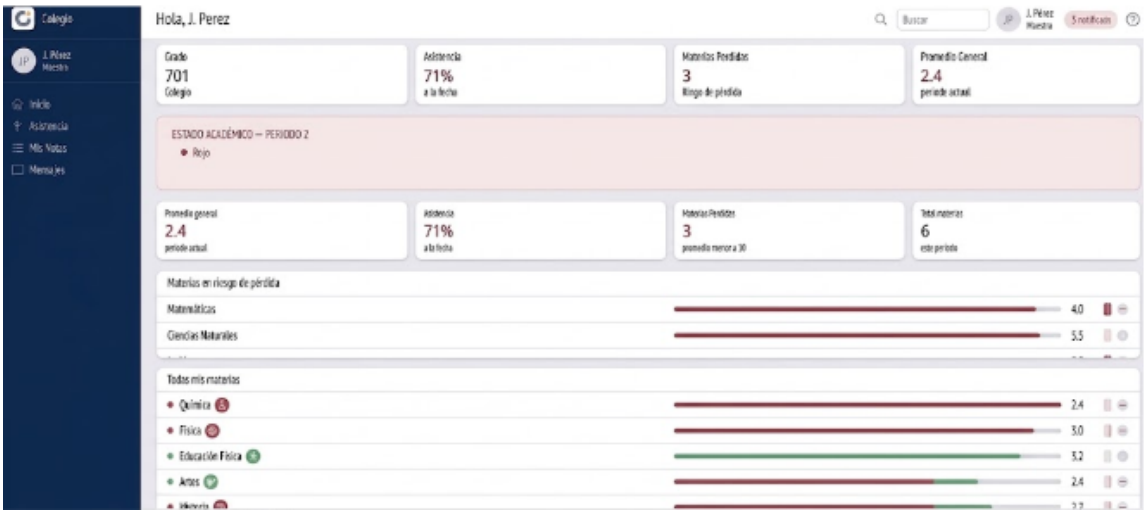


Figure 12: Mockup: Performance Summary